

Appunti Avanzati in C++:

std::map vs std::unordered_map

Introduzione: I Dizionari del C++

Sia `map` che `unordered_map` sono **contenitori associativi**. Servono a memorizzare coppie **Chiave-Valore** (es. Matricola $\rightarrow$ Nome Studente). La regola d'oro è che **le chiavi sono uniche**: non possono esistere due elementi con la stessa chiave.

Nonostante l'interfaccia sia quasi identica, sotto il cofano funzionano in modo opposto.

1. std::map: L'Albero Bilanciato

La `std::map` è implementata internamente come un **Albero Binario di Ricerca Autobilanciante** (solitamente un *Red-Black Tree*).

Questo significa che ogni volta che inserisci un elemento, l'albero si riorganizza per mantenere i rami bilanciati e, soprattutto, **mantiene le chiavi sempre ordinate** (di default in ordine crescente, usando l'operatore `<`).

Caratteristiche e Complessità

- **Ordinamento**: Gli elementi sono *sempre* ordinati per chiave. Se stampi la mappa con un `for`, i dati usciranno in ordine alfabetico/numerico.
- **Ricerca, Inserimento, Rimozione**: Complessità garantita di $O(\log N)$. Veloce, ma non istantanea.
- **Uso ideale**: Quando devi fare "Range Queries" (es. "trovami tutti gli studenti con matricola tra 100 e 200") o quando ti serve scorrere i dati in ordine.

```
1 #include <iostream>
2 #include <map>
3 using namespace std;
4
5 int main() {
6     map<int, string> studenti;
7
8     // Inserimenti sparsi
9     studenti[300] = "Giacomo";
10    studenti[100] = "Aldo";
11    studenti[200] = "Giovanni";
12
13    // Il for stamperà AUTOMATICAMENTE in ordine di chiave: 100, 200, 300!
14    for (const auto& pair : studenti) {
15        cout << pair.first << " -> " << pair.second << endl;
16    }
17    return 0;
18 }
```

2. std::unordered_map: La Hash Table

La `unordered_map` è implementata tramite una **Tabella Hash**.

Quando inserisci una chiave, il C++ usa una "Funzione Hash" per trasformare la chiave (es. una stringa) in un numero intero, che viene usato come indice di un array (chiamato *bucket*).

Caratteristiche e Complessità

- **Nessun Ordinamento:** Gli elementi sono memorizzati in ordine apparentemente casuale. Se usi un ciclo `for`, usciranno alla rinfusa.
- **Ricerca, Inserimento, Rimozione:** Complessità media sbalorditiva di $O(1)$ (istantanea!). Tuttavia, nel caso pessimo (quando ci sono troppe "collisioni hash"), degrada a $O(N)$.
- **Uso ideale:** Quando ti serve la **massima velocità assoluta** per trovare un dato e dell'ordinamento non ti importa nulla.

3. LA TRAPPOLA DA ESAME: L'operatore []

Questa è una delle cause principali di bug (e di voti abbassati). Come si verifica se una chiave esiste in una mappa?

L'approccio SBAGLIATO (da non fare mai per controllare l'esistenza):

```
1 map<string, int> voti;  
2 if (voti["Matematica"] == 30) { ... } // ERRORE LOGICO GRAVE!
```

Perché è sbagliato? Se "Matematica" **non esiste** nella mappa, l'operatore `[]` **LA CREA ALL'ISTANTE** e le assegna un valore di default (0 per gli interi). Oltre a darti una risposta potenzialmente falsa, stai riempiendo la memoria di "spazzatura" a tua insaputa!

L'approccio CORRETTO (usare `find` o `count`):

```
1 // Metodo 1: count() restituisce 1 se esiste, 0 se non esiste  
2 if (voti.count("Matematica") > 0) { ... }  
3  
4 // Metodo 2 (Migliore): find() restituisce un iteratore  
5 auto it = voti.find("Matematica");  
6 if (it != voti.end()) {  
7     cout << "Trovato! Il voto e': " << it->second << endl;  
8 }  
9
```

4. Tabella Riassuntiva Definitiva

Caratteristica	std::map	std::unordered_map
Struttura Interna	Albero Binario (Red-Black Tree).	Tabella Hash (Array di Buckets).
Ordinamento Chiavi	Sì (Ordinati con <code>operator<</code>).	No (Apparentemente casuale).
Complessità Media (Ricerca)	$O(\log N)$ (Logaritmica).	$O(1)$ (Costante - Istantanea).
Complessità Pessima	$O(\log N)$ (Garantita).	$O(N)$ (In caso di collisioni hash severe).
Richiede...	Operatore <code><</code> definito per la chiave.	Funzione <code>hash</code> e operatore <code>==</code> per la chiave.